COMSATS University Islamabad, Vehari Campus, Pakistan

Department of Computer Science

Gamified Learning Mobile App for Programming Fundamentals

A Thesis / Software Requirements Specification (SRS) / Software Design Document (SDD) Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science in Computer Science/Software Engineering

Submitted By

First Candidate Name (Registration Number)
Second Candidate Name (Registration Number)

Supervisor

Syeda Zoupash Zahra

Session: Spring 2022-2026

Dedication

I dedicate this work to my parents, whose unwavering support, sacrifices, and prayers have been the foundation of my success. Their encouragement, patience, and belief in my abilities have guided me throughout this journey.

I also dedicate this to my teachers and mentors who inspired me to pursue knowledge with dedication and integrity.

Acknowledgement

All praise and gratitude are due for the successful completion of this project. The authors would like to express their sincere appreciation to their supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this thesis. The supervision and mentorship provided significant support in completing this work successfully.

Special thanks are extended to the faculty members of the Department of Computer Science for their academic support and encouragement during the degree program.

The authors are also deeply grateful to their parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength.

Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project.

Certificate of Approval

This is to certify that the Final Year Project titled **"Project Title Here"** has been developed by **Student 1 Name (Student 1 Registration number)** and **Student 2 Name (Student 2 Registration number)** under the supervision of **Supervisor Name** and that in his/her opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Computer Science / Software Engineering.

_____

Supervisor

_____

External Examiner

_____

Head of Department
Department of Computer Science

Plagiarism and AI Content Certificate

Date: _____

It is hereby certified that the similarity index (as generated by the Turnitin Originality Report) of the [**Report Type: SRS / SDD / Thesis / Final Year Project Report**] submitted by:

- **Student 1 Name (Registration Number)**
- **Student 2 Name (Registration Number)**

titled:

"[Project Title Here]"

is [**XX%**] which is within the permissible limits as defined by the Higher Education Commission (HEC) rules and regulations. The similarity index report is attached herewith.

Furthermore, the AI-generated content detection index (if applicable as per institutional policy) is reported as [**XX%**]. The report has been reviewed, and the content is found to comply with university guidelines regarding ethical use of Artificial Intelligence tools.

The report is recommended for submission.

Supervisor Name: _____

Designation: _____

Signature: _____

Department: Department of Computer Science

University: COMSATS University Islamabad, Vehari Campus

Executive Summary

Provide a concise overview of the project, highlighting:

• Background and motivation • Problem being addressed • Proposed solution • Key features of the system • Technologies used • Expected impact or contribution

The executive summary should clearly communicate the purpose and scope of the project in 1–2 pages.

Abbreviations

Abbreviation	Full Form
SRS	Software Requirements Specification
SDD	Software Design Document
API	Application Programming Interface
UI	User Interface
DB	Database

Table of Contents

List of Tables

List of Figures

Chapter 1
Introduction

1.1 Introduction

Writing Guidelines (400–600 words)

- Provide background of the selected problem domain.
- Describe current industry practices and existing solutions.
- Identify key challenges, inefficiencies, or gaps.
- Justify the need for the proposed system.
- Briefly introduce your proposed system and its purpose.
- Maintain a formal academic tone and avoid informal language.

Replace this guideline text with your final written content before submission.

1.2 Vision Statement

Writing Guidelines (150–250 words)

- Clearly define the target users or stakeholders.
- State the core problem the system intends to solve.
- Describe the long-term vision and expected impact.
- Highlight innovation and uniqueness.
- Focus on strategic impact rather than implementation details.

Insert your Vision Statement here.

1.3 Related System Analysis

Writing Guidelines (500–700 words)

- Discuss 2–4 existing systems relevant to your project.
- Describe their main features and functionalities.

- Critically evaluate their weaknesses or limitations.
- Compare them with your proposed solution.
- Clearly justify how your system improves upon existing approaches.

Insert detailed analytical discussion here.

Table 1.1: Comparative Analysis of Existing Systems

Application Name	**Identified Weaknesses**	**Proposed Improvements**
System A	• Weakness 1 • Weakness 2	• Improvement 1 • Improvement 2
System B	• Weakness 1 • Weakness 2	• Improvement 1 • Improvement 2

1.4 Project Deliverables

Writing Guidelines (400–600 words)

- Clearly list all tangible outputs of the project.
- Explain the purpose and functionality of each deliverable.
- Identify system modules developed.
- Include documentation artifacts (SRS, SDD, test cases, etc.).
- Mention deployment or implementation outputs if applicable.

List of Deliverables:

1. Deliverable 1 – Description of functionality and purpose.
2. Deliverable 2 – Description of functionality and purpose.
3. Deliverable 3 – Description of functionality and purpose.

1.5 System Limitations / Constraints

Writing Guidelines (200–300 words)

- Identify technical limitations of the system.
- Mention hardware or software dependencies.
- Discuss time, budget, or resource constraints.
- Include scalability or deployment limitations if applicable.

Example Format:

- Limitation 1 – Brief explanation.
- Limitation 2 – Brief explanation.
- Limitation 3 – Brief explanation.

1.6 Tools and Technologies

Writing Guidelines (200–300 words)

- Justify the selection of each major technology.
- Mention backend, frontend, and database tools.
- Describe the development environment.
- Provide brief technical justification for choices.

Table 1.2: Tools and Technologies Used in the Project

Tool / Technology	**Version**	**Purpose**
Tool 1	Version	Purpose description
Tool 2	Version	Purpose description
Tool 3	Version	Purpose description

1.7 Relevance to Course Modules

Writing Guidelines (400–600 words total)

- Demonstrate integration of academic knowledge into practical implementation.
- Reflect on how theoretical concepts were applied.
- Connect specific course modules to project components.
- Highlight achievement of learning outcomes.

1.7.1 Programming Fundamentals

Explain (80–120 words) how programming logic, object-oriented principles, modular design, and code structuring were applied in the project.

1.7.2 Data Structures and Algorithms

Explain (80–120 words) how data handling, searching, sorting, optimization techniques, or algorithms were utilized.

1.7.3 Database Management Systems

Explain (80–120 words) how database design, normalization, relationships, and queries were implemented.

1.7.4 Software Engineering

Explain (80–120 words) how SDLC models, UML diagrams, testing strategies, documentation, and requirement engineering were applied.

1.7.5 Other Relevant Courses

Explain (80–120 words) how additional subjects (e.g., AI, Networking, Web Development, Cybersecurity, HCI, etc.) contributed to the project implementation.

1.8 Chapter Summary

This chapter introduced the background and context of the proposed system, highlighting existing challenges and the need for improvement within the selected problem domain. The vision and strategic objectives of the system were outlined to establish its intended impact and innovation. A comparative analysis of related systems identified key limitations in current solutions. Project deliverables, system constraints, selected technologies, and academic relevance were also discussed.

This chapter establishes the foundation for the detailed problem definition presented in the next chapter.

Chapter 2
Problem Definition

2.1 Problem Statement

Writing Guidelines (100–150 words)

Clearly define the core problem that the proposed system aims to solve. Explain existing challenges, inefficiencies, limitations, or gaps in the current system or process. Identify affected stakeholders such as users, organizations, or administrators. Highlight why this problem is significant in technical, operational, or societal terms. Avoid describing the proposed solution here — focus strictly on defining the problem in a clear, logical, and evidence-based manner.

Replace this text with your finalized Problem Statement before submission.

2.2 Proposed Solution Overview

Writing Guidelines (100–150 words)

Provide a high-level conceptual description of the proposed system. Explain how it addresses the issues identified in the Problem Statement. Mention major features, innovations, or technologies used. Describe the value proposition and expected improvements in efficiency, usability, performance, automation, or transparency. Keep the explanation conceptual and avoid detailed implementation or technical architecture at this stage.

Insert the overview of your proposed solution here.

2.3 Objectives of the Proposed System

Writing Guidelines

List clear, measurable, and verifiable objectives. Each objective must:

- Begin with an action verb (e.g., Develop, Provide, Improve, Ensure).
- Be specific and testable.
- Directly align with the Problem Statement.
- Be achievable within project constraints.

Business Objectives (BO):

- BO-1: Develop a system that ...
- BO-2: Improve ...

- BO-3: Provide ...
- BO-4: Ensure ...
- BO-5: Automate ...

2.4 Scope of the System

Writing Guidelines (100–150 words)

Define the boundaries of the project clearly. Specify what the system will include and what it will not include. Identify the target users, supported platforms (web, mobile, desktop), and core functionalities. Clarify operational limits and project coverage. Distinguish scope from limitations — scope defines coverage, while limitations define constraints.

Insert scope description here.

2.5 System Architecture and Modules

Writing Guidelines (100–150 words)

Provide an overview of the system architecture. Specify whether the system follows client-server, three-tier, MVC, microservices, or another architectural pattern. Explain why modular design is used. Describe how modules interact and exchange data. A high-level system architecture diagram should be included in this section.

Insert architecture overview here.

2.5.1 Module 1: Profile Management

Description (80–120 words)

Explain the purpose of this module. Describe user registration, authentication, authorization, and profile management features. Specify supported user roles.

Functional Requirements:

- FE-1: The system shall allow users to register using valid credentials.
- FE-2: The system shall authenticate users before granting access.

2.5.2 Module 2: Communication Module

Description (80–120 words)

Describe how users interact within the system. Mention messaging, notifications, scheduling, or real-time communication capabilities.

Functional Requirements:

- FE-1: The system shall enable secure messaging between users.

- FE-2: The system shall generate real-time notifications.
- FE-3: The system shall maintain communication logs.

2.5.3 Module 3: Core Functional Module

Description (80–120 words)

Explain the primary operational features that deliver the core value of the system. This module should directly address the problem identified in Section 2.1.

Functional Requirements:

- FE-1: The system shall process user requests efficiently.
- FE-2: The system shall store and retrieve data accurately.
- FE-3: The system shall generate relevant outputs based on user input.

2.5.4 Module 4: Management / Admin Module

Description (80–120 words)

Explain administrative controls, monitoring dashboards, reporting tools, and configuration management features.

Functional Requirements:

- FE-1: The system shall allow administrators to manage user accounts.
- FE-2: The system shall generate system usage reports.

2.5.5 Module 5: Advanced / Additional Features

Description (80–120 words)

Describe advanced capabilities such as AI-based recommendations, data analytics, automation, payment gateway integration, or external API integration.

Functional Requirements:

- FE-1: The system shall integrate with external APIs securely.
- FE-2: The system shall provide analytics-based insights.

2.6 Assumptions and Dependencies

Writing Guidelines (80–120 words)

List assumptions made during system design. Identify dependencies on external systems, APIs, hardware, or third-party services.

- The system assumes users have stable internet connectivity.
- The system depends on third-party API availability.
- The system requires compatible hardware/software environments.

2.7 Chapter Summary

This chapter defined the core problem addressed by the proposed system and outlined the conceptual solution approach. Clear objectives were established to ensure alignment with identified challenges. The system scope and architectural overview were presented to define structural boundaries and major modules. Key assumptions and dependencies were also documented to clarify operational constraints.

This chapter provides the problem foundation and sets the direction for the detailed software design presented in the next chapter.

Chapter 3
Requirement Analysis

3.1 Introduction to Requirement Analysis

This chapter presents a structured and formal specification of system requirements for the proposed project. The purpose of this chapter is to clearly define what the system shall accomplish before proceeding to the system design phase. The requirements documented in this chapter serve as a contractual foundation for implementation, validation, and system testing.

The requirement analysis includes requirement elicitation techniques, identification of stakeholders, use case modeling, functional requirements, non-functional requirements, and external interface requirements. All requirements are written in measurable and testable format using standard requirement engineering principles.

3.2 Requirement Elicitation Techniques

This section describes the techniques used to gather and refine system requirements. Requirement elicitation ensures alignment between stakeholder expectations and system capabilities.

Requirements for this project were identified using the following techniques:

- Stakeholder interviews to capture user expectations and constraints.
- Literature review of similar systems and research contributions.
- Analysis of existing solutions to identify functional gaps.
- Dataset examination and feasibility study (for ML-based systems).
- Observation of workflow processes (if applicable).

3.3 User Classes and Characteristics

This section identifies different user categories interacting with the system. Each user class is defined along with their responsibilities, access privileges, and technical proficiency level. This classification supports role-based access control and requirement mapping.

3.4 System Overview

The proposed system operates within a structured architectural environment designed to support modular interaction between user interfaces, processing logic, and data storage. The system supports role-based interactions and ensures secure communication between internal modules and external services.

Table 3.1: User Classes and Characteristics

User Class	Description	Technical Level
Administrator	Manages system configuration, user accounts, monitoring, and reports.	Advanced
End User	Uses the system to perform core functionalities such as data input, search, prediction, or transactions.	Basic
Service Provider	Provides domain-specific services within the system (if applicable).	Intermediate
External System	APIs or third-party systems interacting with the application.	Technical

The overall architecture may follow client-server, three-tier, microservices, or machine learning pipeline architecture depending on project type. The system boundary includes all internal modules, user interfaces, data processing components, and integrated third-party services.

3.5 Use Case Model

The use case model represents high-level system interactions between actors and system functionalities. It defines system boundaries and captures functional behavior from a user perspective.

3.5.1 Use Case Diagram

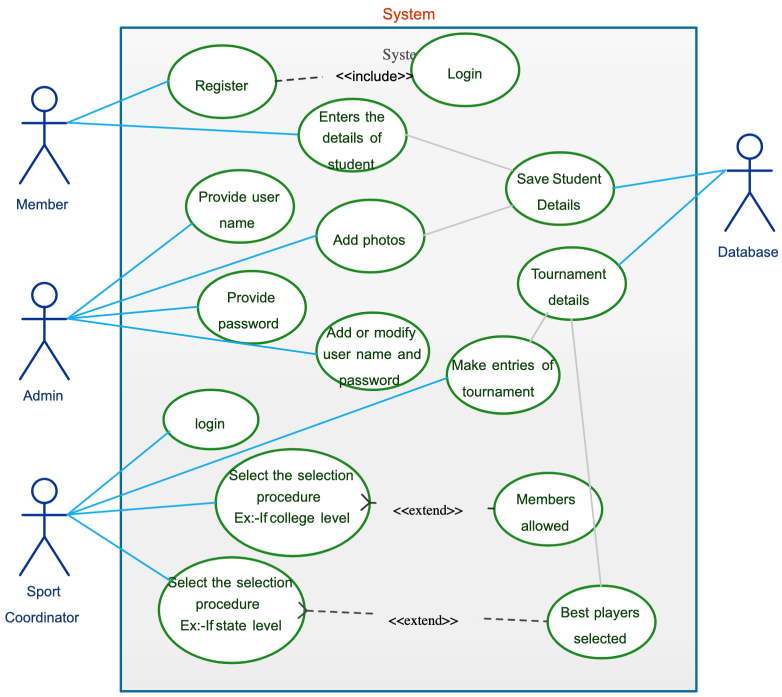

Figure 3.1: Use Case Diagram of Proposed System

The above diagram illustrates the interaction between actors and major system functionalities. Include relationships such as *include* and *extend* where applicable.

3.5.2 Use Case Descriptions

This section provides detailed tabular specifications of major use cases. These use cases form the basis for deriving functional requirements.

3.5.2.1 Module 1: Core System Functionality

Below are the detailed use cases for Module 1.

Table 3.2: UC-1.1: User Registration

Use Case ID	UC-1.1
Use Case Name	User Registration
Actors	End User
Description	This use case describes how a user registers into the system.
Trigger	User selects the "Sign Up" option.
Preconditions	User is not already registered.
Postconditions	User account is successfully created.
Normal Flow	1. User enters required information. 2. System validates input fields. 3. System stores data in database. 4. Confirmation message is displayed.
Alternative Flow	If validation fails, the system displays appropriate error messages.

3.5.3 Event-Response Table

The event-response table defines how the system reacts to significant internal and external events.

Table 3.3: Event-Response Table

Event	**Source**	**System Response**
User Login Attempt	End User	Validate credentials and grant or deny access.
Data Upload	User/Admin	Store dataset in database and validate file format.
Prediction Request	End User	Execute trained model and display prediction results.
Payment Submission	Customer	Process payment via secure gateway and confirm transaction.

3.6 Functional Requirements

Functional requirements define the core services and behaviors that the system shall provide. These requirements are derived directly from the use case model presented in Section 3.5. Each requirement is uniquely identified, measurable, and written using formal specification language.

All functional requirements follow IEEE-style structure and are categorized by system modules.

3.6.1 Module 1: User Management

3.6.1.1 FR-1.1 User Registration

Table 3.4: Functional Requirement – User Registration

Identifier	FR-1.1
Title	User Registration
Requirement	The system shall allow users to create an account by providing valid credentials.
Source	Stakeholder Requirement
Rationale	Enables authorized access to system services.
Dependencies	Database Connectivity
Priority	High

3.6.2 Module 2: Core Processing / ML / Business Logic

3.6.2.1 FR-2.1 Core Processing

Table 3.5: Functional Requirement – Core Processing

Identifier	FR-2.1
Title	Execute Core Processing Logic
Requirement	The system shall process user inputs and generate appropriate outputs based on defined algorithms or business logic.
Source	System Requirement Specification
Rationale	Enables delivery of the primary system value proposition.
Dependencies	Input Validation Module
Priority	High

Table 3.6: Functional Requirement – Reporting

Identifier	FR-3.1
Title	Generate Reports
Requirement	The system shall generate system reports and analytics based on stored data.
Source	Business Requirement
Rationale	Supports monitoring, analysis, and decision-making.
Dependencies	Database and Processing Module
Priority	Medium

3.6.3 Module 3: Reporting and Analytics

3.6.3.1 FR-3.1 Report Generation

3.7 Non-Functional Requirements

Non-functional requirements define quality attributes that describe how the system performs. These requirements are measurable and support system reliability, usability, performance, security, and maintainability.

3.7.1 Performance Requirements

- PER-1: The system shall load the main dashboard within 3 seconds under normal operating conditions.
- PER-2: The system shall support concurrent users as defined in the project scope.
- PER-3: For ML-based systems, prediction responses shall be generated within acceptable latency.
- PER-4: The system shall handle peak load without significant degradation in response time.

3.7.2 Security Requirements

- SEC-1: The system shall implement secure authentication mechanisms.
- SEC-2: Passwords shall be encrypted using secure hashing algorithms.
- SEC-3: Sensitive data shall be transmitted using HTTPS protocol.
- SEC-4: Role-based access control shall be enforced.
- SEC-5: Security vulnerabilities shall be periodically assessed.

3.7.3 Reliability Requirements

- REL-1: The system shall maintain at least 99% uptime.
- REL-2: System failures shall not result in data loss.
- REL-3: Backup and recovery mechanisms shall be implemented.

3.7.4 Usability Requirements

- USE-1: The interface shall be intuitive and easy to navigate.
- USE-2: Clear and meaningful error messages shall be displayed.
- USE-3: The system shall follow standard UI/UX design principles.
- USE-4: The application shall be responsive across devices.

3.7.5 Scalability Requirements

- SCA-1: The system shall support increasing number of users.
- SCA-2: The system architecture shall allow horizontal or vertical scaling.
- SCA-3: Database design shall support large datasets.

3.7.6 Maintainability Requirements

- MAIN-1: The system shall follow modular architecture principles.
- MAIN-2: Source code shall be documented clearly.
- MAIN-3: Future updates shall be integrated without major system restructuring.

3.8 External Interface Requirements

This section defines how the system interacts with external entities including users, hardware devices, software systems, and communication networks.

3.8.1 User Interface Requirements

- UI-1: The system shall provide a graphical user interface.
- UI-2: Standard font sizes shall ensure readability (12–14pt).
- UI-3: Layout shall be responsive across devices.
- UI-4: Navigation shall remain consistent across modules.

3.8.2 Hardware Interface Requirements

- HI-1: The system shall operate on standard computing devices.
- HI-2: For mobile applications, it shall support Android/iOS platforms.
- HI-3: For ML systems, GPU support may be required for training processes.

3.8.3 Software Interface Requirements

- SI-1: The system shall interact with a relational or non-relational database.
- SI-2: Third-party APIs shall be securely integrated.
- SI-3: Payment gateways (if applicable) shall be securely integrated.
- SI-4: ML systems shall integrate with frameworks such as TensorFlow or PyTorch.

3.8.4 Communication Interface Requirements

- CI-1: The system shall use HTTPS protocol for secure communication.
- CI-2: RESTful APIs shall be used for backend communication.
- CI-3: Real-time communication (if applicable) shall use WebSockets.
- CI-4: For distributed systems, secure cloud communication shall be ensured.

3.9 Assumptions and Dependencies

This section lists assumptions made during requirement analysis and system design planning.

- The system assumes stable internet connectivity for online deployment.
- Third-party services are assumed to remain available and functional.
- Users are assumed to possess basic digital literacy.
- Required hardware resources are assumed to be available during deployment.

3.10 Requirement Traceability Matrix

This matrix maps project objectives to functional requirements to ensure alignment and completeness.

Table 3.7: Requirement Traceability Matrix

Objective	**Use Case**	**Functional Requirement**
BO-1	UC-1.1	FR-1.1
BO-2	UC-2.1	FR-2.1
BO-3	UC-3.1	FR-3.1

Chapter Summary

This chapter presented a structured requirement analysis of the proposed system. It defined requirement elicitation techniques, user classes, use case models, functional requirements, non-functional requirements, and external interface requirements. A traceability matrix was included to ensure alignment between objectives and implementation requirements. These requirements form the formal foundation for the system design presented in the next chapter.

Chapter 4
Software Design

4.1 Introduction to Software Design

This chapter presents the detailed software design of the proposed system. It translates the requirements identified in Chapter 3 into structured architectural, behavioral, and data models. The purpose of this chapter is to provide a clear blueprint for system implementation, ensuring maintainability, scalability, and modular development.

This chapter includes architectural design, UML models, data flow models, and database design applicable to machine learning systems, web/mobile applications, and other software engineering domains.

4.2 Design Methodology and Software Process Model

4.2.1 Design Methodology

This project follows a structured design methodology. Depending on the project type, one of the following approaches may be adopted:

- Object-Oriented Design (OOD)
- Model-View-Controller (MVC)
- Layered Architecture
- Microservices Architecture
- Client-Server Architecture
- Data-Centric Design (for ML systems)

The selected methodology ensures modularity, loose coupling, reusability, and scalability.
Guidelines:

- Clearly justify the selected design approach.
- Explain why it suits your domain (ML, Web, Mobile, etc.).
- Mention design principles followed (SOLID, DRY, Separation of Concerns).

4.2.2 Software Process Model

The project follows the **(Agile / Waterfall / Spiral / Incremental)** model.

Guidelines:

- Explain why this process model was selected.
- Describe iteration cycles or development phases.
- Explain how testing and validation were integrated.

4.3 System Overview

This section provides a high-level overview of system components and interactions with external entities.

Include:

- Context Diagram
- External Actors
- Major Subsystems

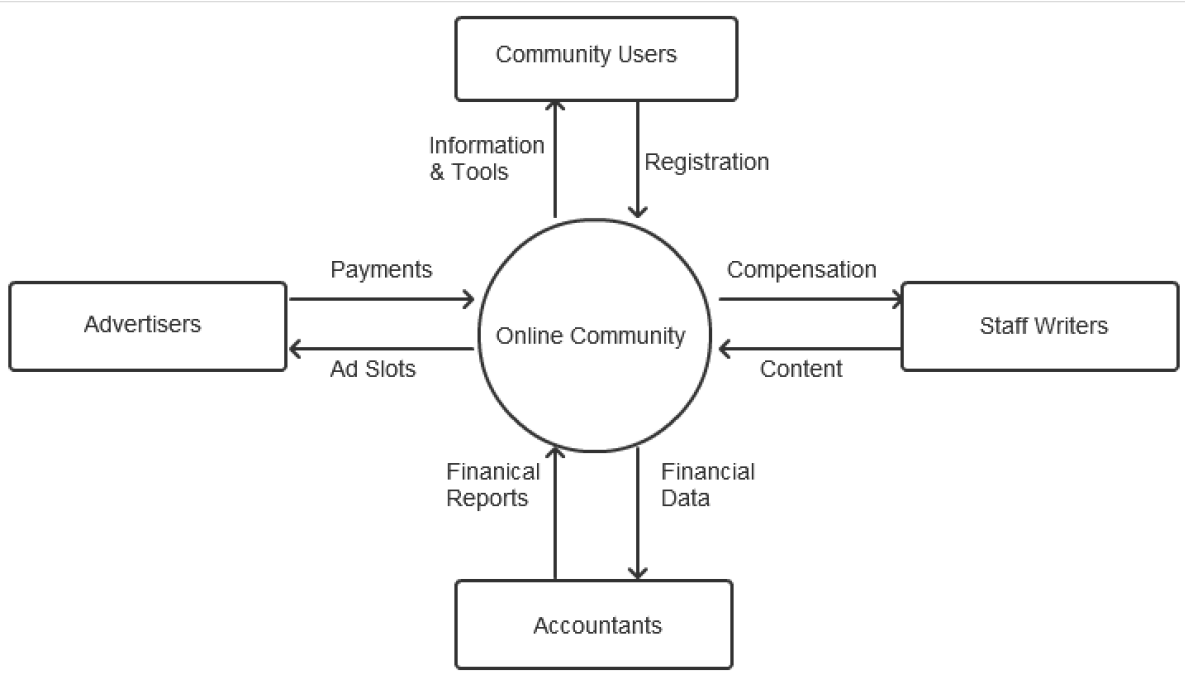

Figure 4.1: Sample Context Diagram of Proposed System

Guidelines:

- Show system boundary.
- Include users, admin, third-party APIs, databases, ML engines, payment gateways, etc.
- Keep diagram simple and high-level.

4.4 Architectural Design

This section describes the high-level architecture of the system.

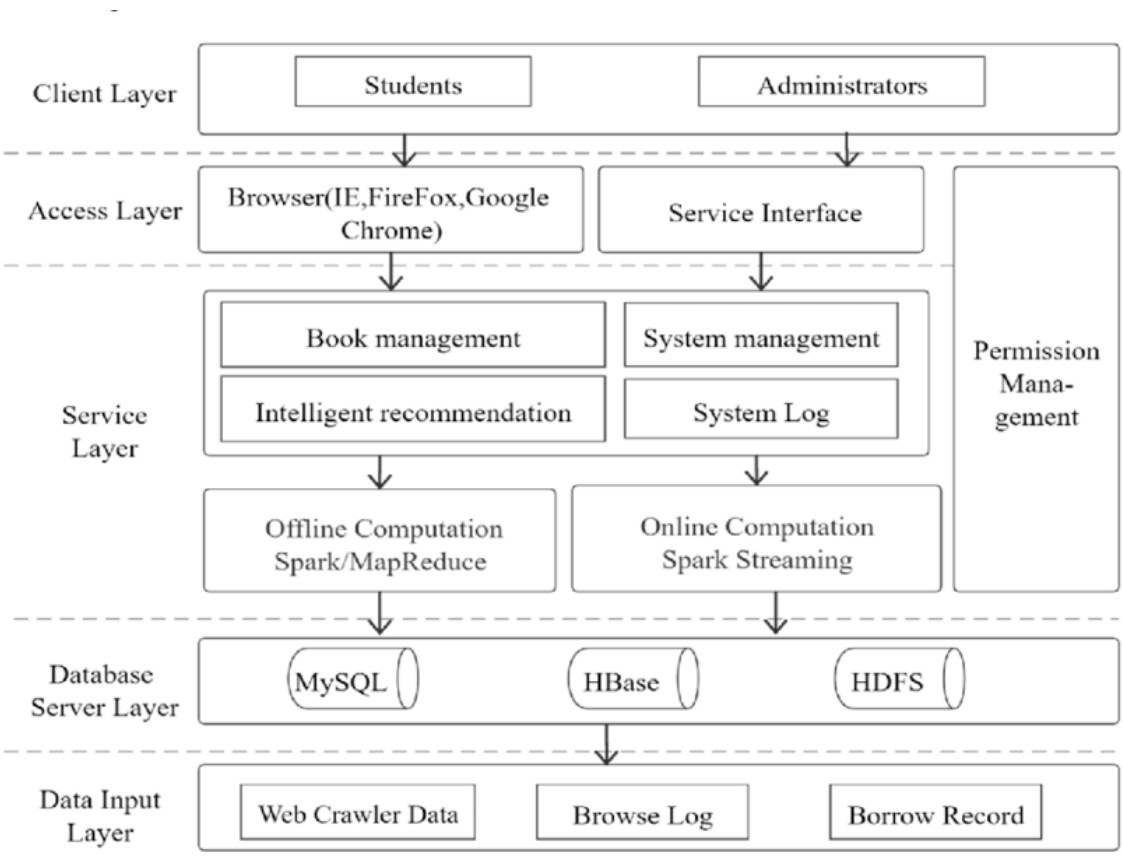

Figure 4.2: Sample System Architecture Diagram of Proposed System

Guidelines:

- Divide system into layers (Presentation, Business Logic, Data Layer).
- For ML projects, include:
  - Data Preprocessing Module
  - Model Training Module
  - Prediction Engine
  - Evaluation Module
- For Web/Mobile:
  - Frontend
  - Backend
  - API Layer

  - Database

- Explain interaction between components.

4.5 Design Models

This section presents detailed design models of the system. These models describe system behavior, structure, and interactions using UML diagrams. The models help developers understand system workflow before implementation.

4.5.1 Activity Diagrams

Activity diagrams represent the workflow of different modules of the system. Each core module must have a separate activity diagram.

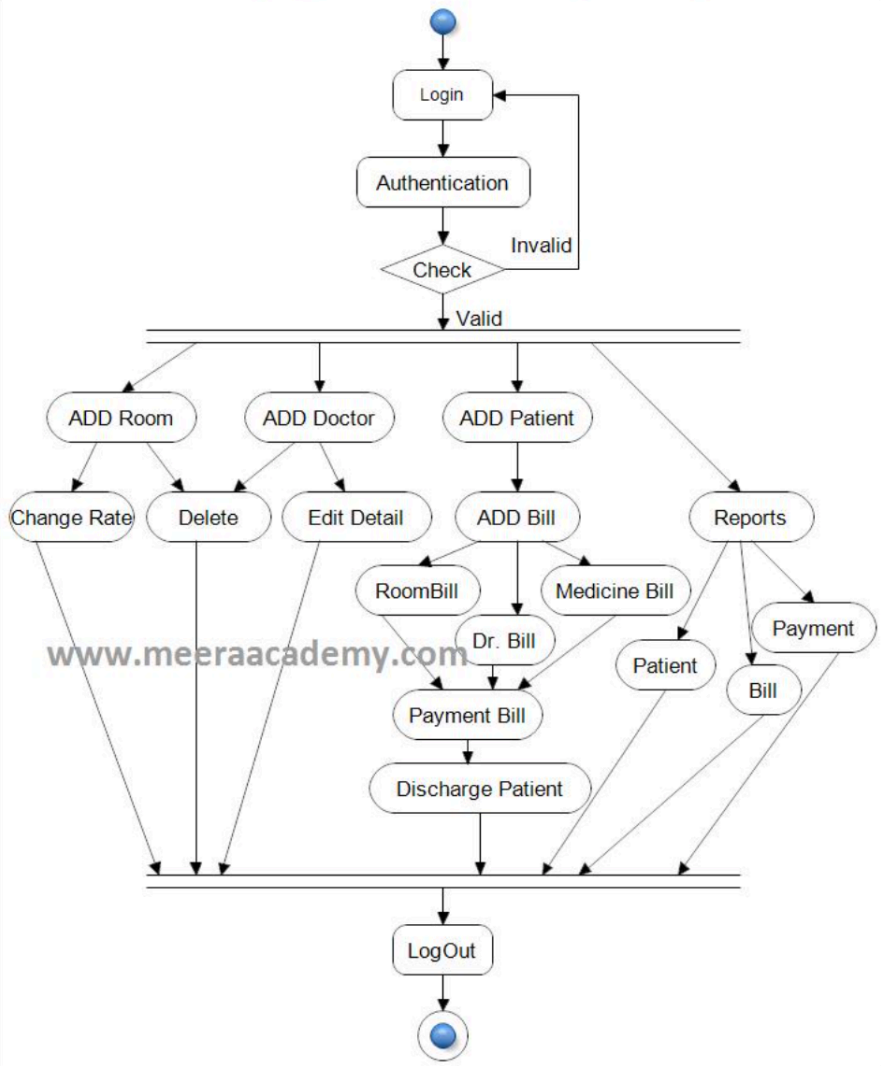

Figure 4.3: Sample Activity Diagram

Guidelines:

- Start with user interaction.
- Include decision nodes.
- Show validation checks.
- Include database/model interaction.
- Clearly indicate start and end states.

4.5.1.1 Module 1: User Management

Description: This diagram illustrates user registration, login, authentication, and profile management processes.

Figure 4.X: Activity Diagram – User Module

4.5.1.2 Module 2: Core Functional Module

(Examples depending on domain)

For ML: - Data Upload - Model Training - Prediction Generation

For Web/Mobile: - Order Processing - Search System - Booking System

Figure 4.X: Activity Diagram – Core Module

4.5.1.3 Module 3: Admin Module

Figure 4.X: Activity Diagram – Admin Operations

Include: - Monitoring - Reports - Data Management - User Control

4.5.2 Class Diagram

The class diagram shows the static structure of the system.

Figure 4.X: Class Diagram

Guidelines:

- Include classes, attributes, and methods.
- Show relationships (Association, Aggregation, Inheritance).
- For ML systems include:
  - Dataset Class
  - Model Class
  - Evaluation Class
- For Web/Mobile:
  - User

- Admin
- Order/Transaction
- Service

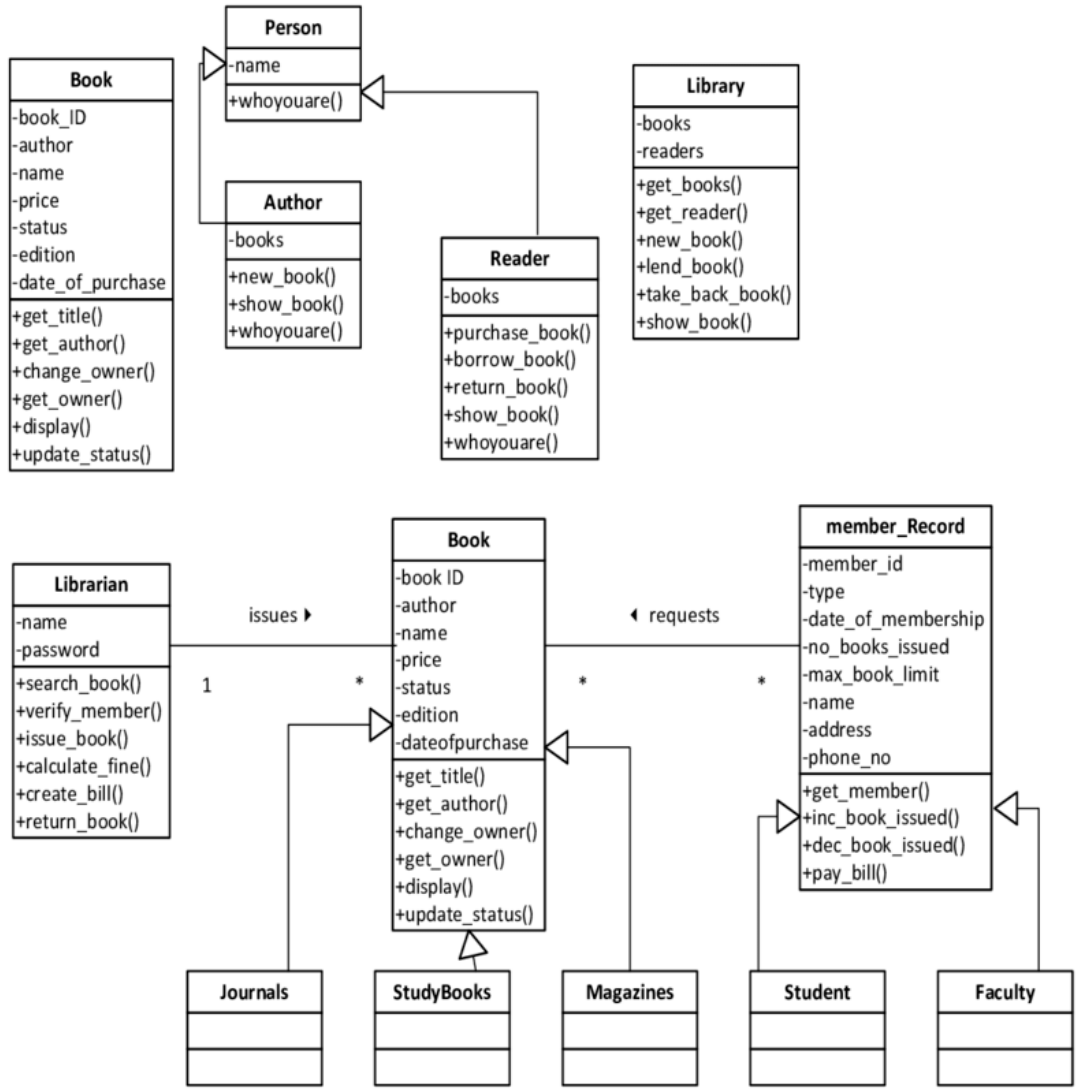

Figure 4.4: Sample Class Diagram

Include explanation after diagram describing:

- Key classes
- Relationships
- Responsibilities

4.5.3 Sequence Diagrams

Sequence diagrams show interaction between objects over time.

Guidelines:

- Include actor on left.
- Show request-response flow.
- Include system components.
- Show database/API interaction.

4.5.3.1 Sequence Diagram – User Login

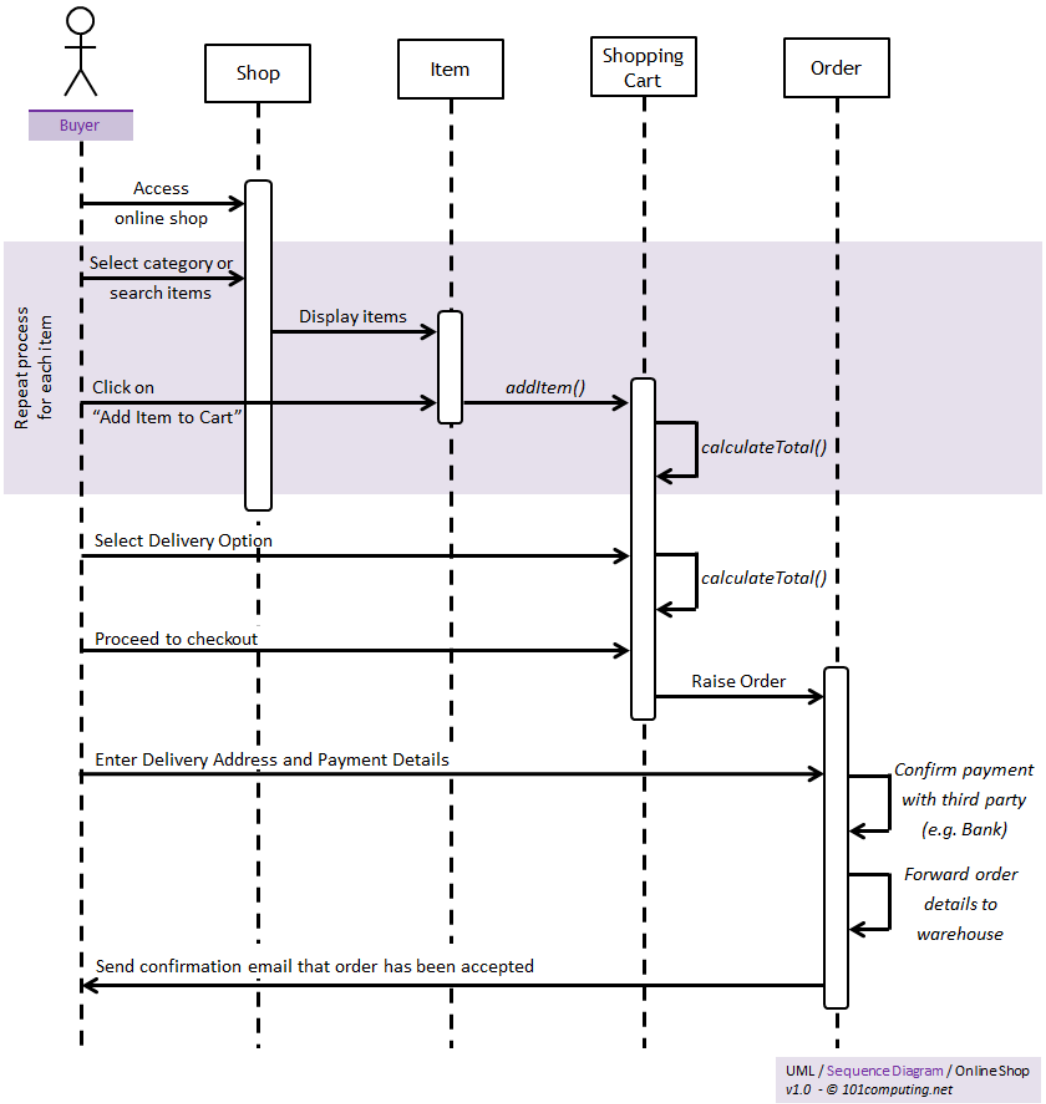

Figure 4.5: Sample Sequence Diagram

Describe: - User sends credentials - System validates - Database checks - Response returned

4.5.3.2 Sequence Diagram – Core Operation

Examples:

For ML: - Upload Dataset → Preprocess → Load Model → Predict → Display Result
For Web: - Add to Cart → Checkout → Payment Gateway → Confirmation

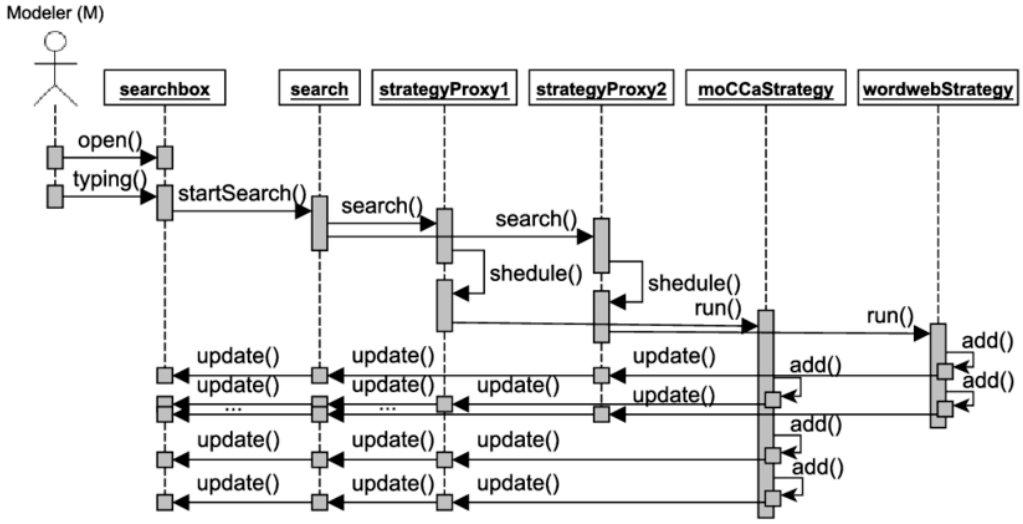

Figure 4.6: Sample Sequence Diagram - 2

4.5.4 State Transition Diagrams

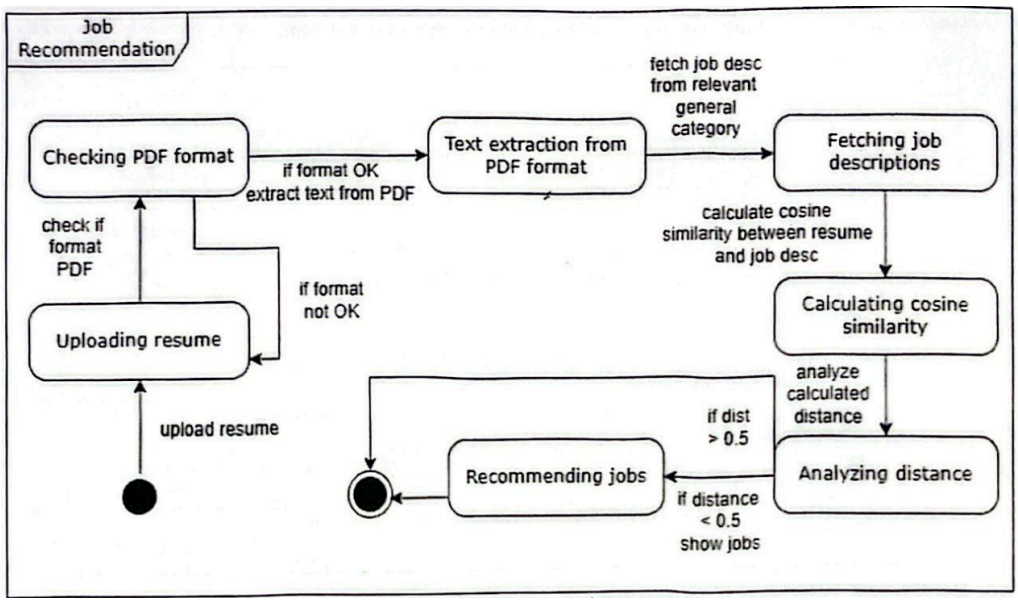

Figure 4.7: Sample State Transition Diagram

State diagrams show how system or object changes states over time.

Examples:

- User Account States (Registered → Active → Suspended → Deleted)
- Order States (Created → Paid → Shipped → Delivered)
- ML Model States (Initialized → Training → Trained → Deployed)

Guidelines:

- Clearly define states.
- Show triggering events.
- Include transitions with conditions.

4.6 Data Flow Diagrams

Data Flow Diagrams (DFD) describe how data moves within the system. Different levels of DFD provide increasing detail.

4.6.1 Level 1 Data Flow Diagram

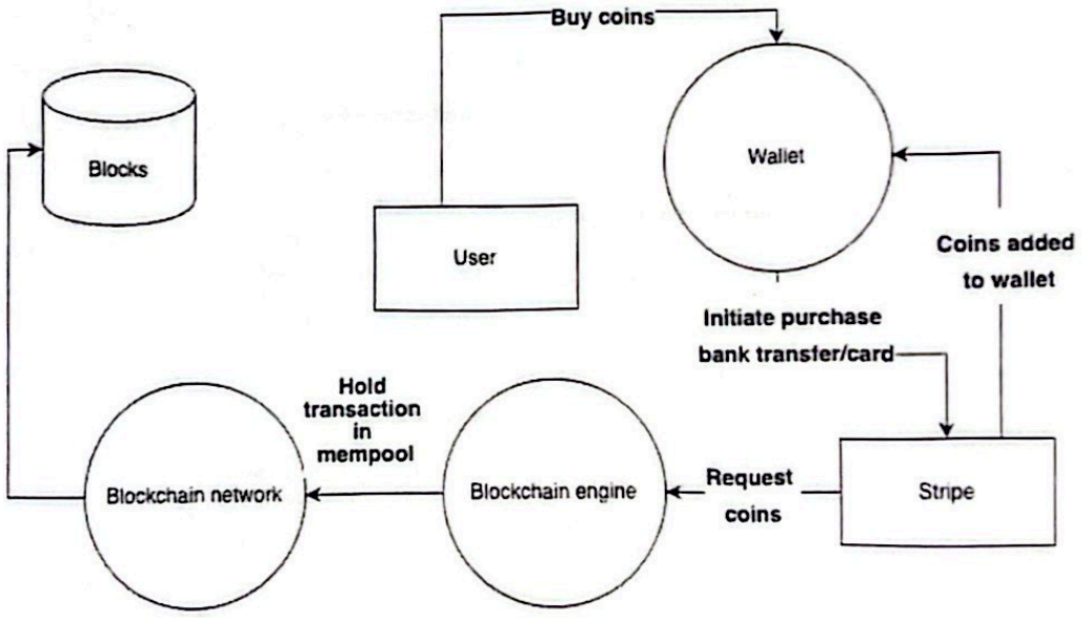

Figure 4.8: Sample Level 1 DFD

Description: Level 1 DFD decomposes the main system into major sub-processes.
Include:

- Authentication Module

- Core Functional Module
- Data Processing Module
- Admin Module
- Database/Data Store

Guidelines:

- Show internal processes.
- Show data stores.
- Maintain data flow consistency with Level 0.

4.6.2 Level 2 Data Flow Diagram

Description: Level 2 DFD further decomposes one complex module from Level 1.

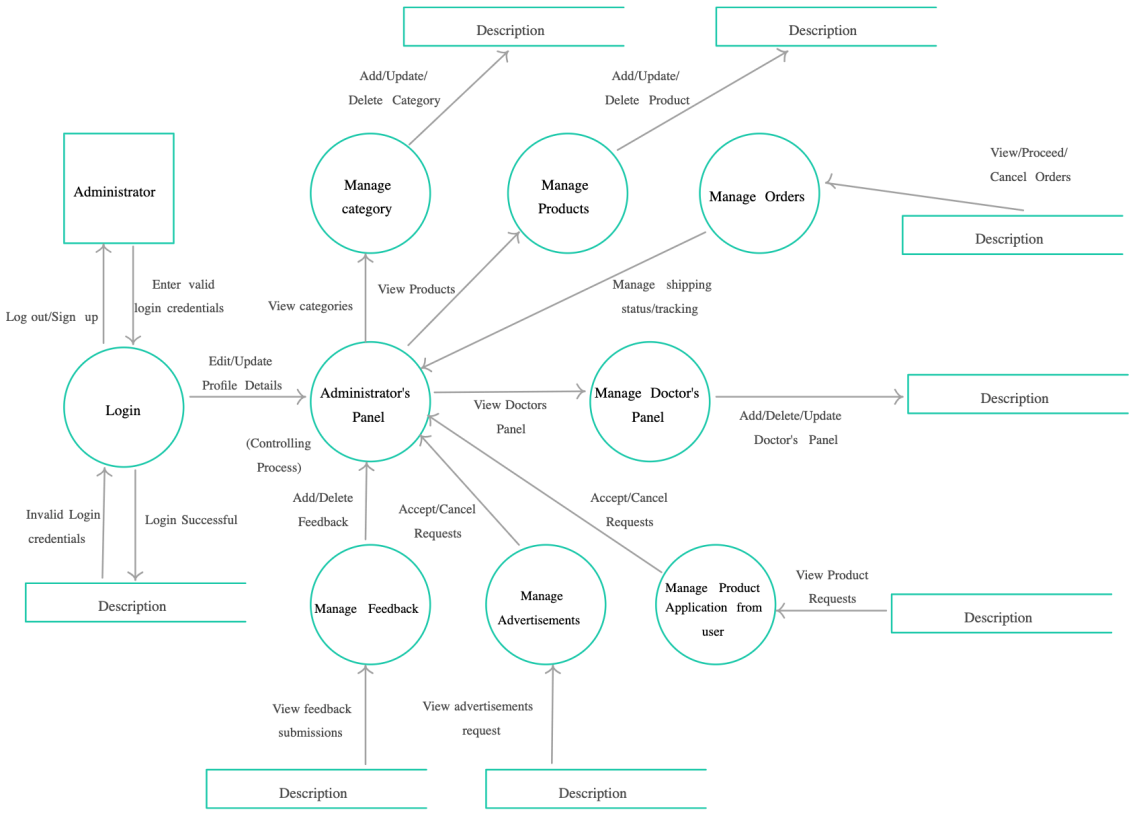

Figure 4.9: Sample Level 2 DFD

Examples (Domain-based):
For Machine Learning:

- Data Upload
- Data Preprocessing
- Model Training
- Prediction Generation

For Web/Mobile:

- Cart Processing
- Payment Handling
- Booking Workflow

For Blockchain:

- Transaction Validation
- Consensus Mechanism
- Block Creation

4.7 Data Design

This section describes how data is structured, stored, and managed.
Guidelines:

- Define database type (SQL / NoSQL).
- Explain schema structure.
- Describe indexing strategy.
- Mention security mechanisms.
- Discuss scalability considerations.

4.7.1 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) represents the logical database structure of the system.
Guidelines:

- Clearly identify all entities (tables/collections).
- Highlight Primary Keys (PK).
- Highlight Foreign Keys (FK).
- Show relationship cardinality (1:1, 1:N, M:N).

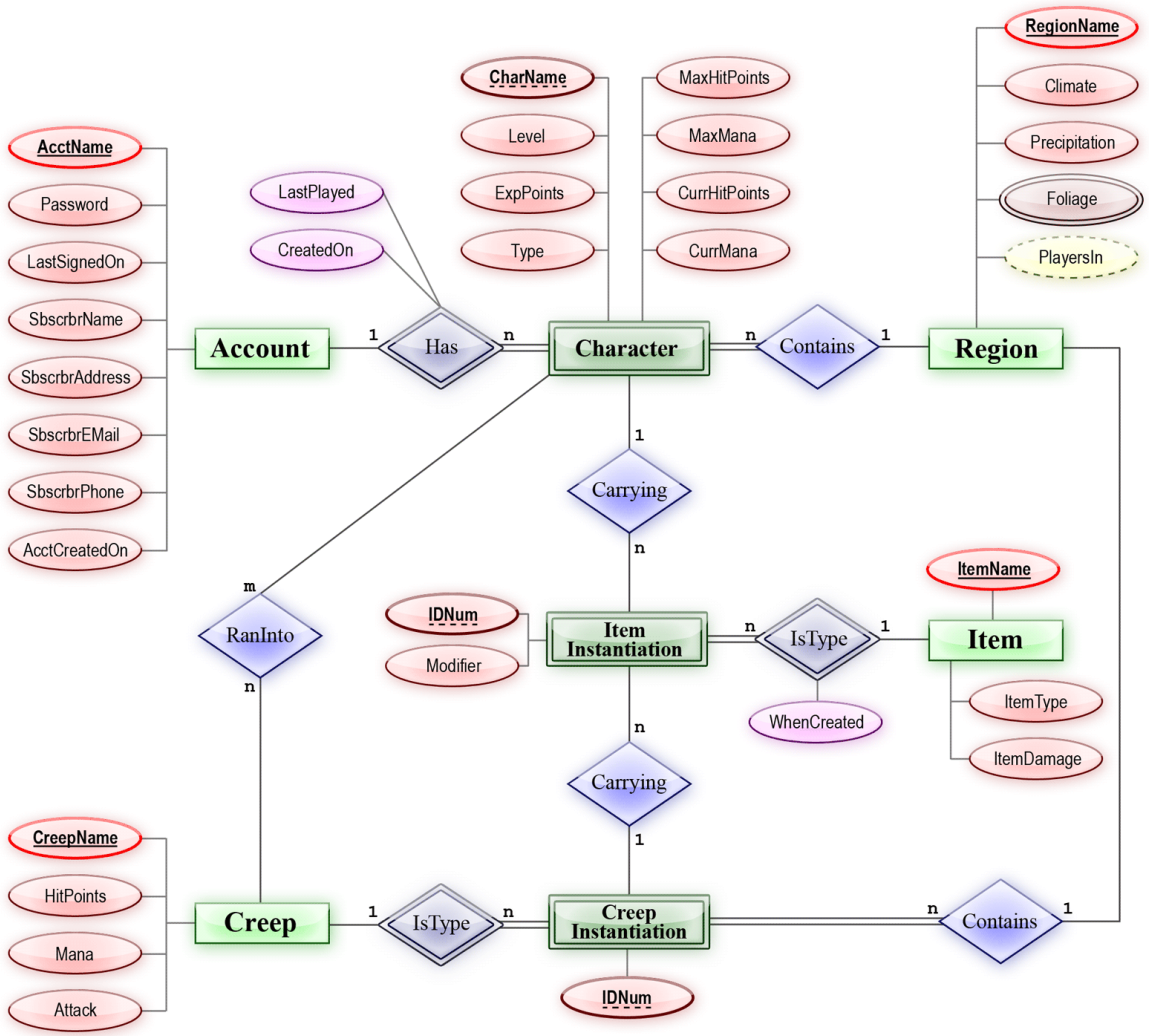

Figure 4.10: Sample Entity Relation Diagram

- Maintain consistency with database tables defined earlier.

Include in ERD:

- User Entity
- Core Functional Entities (domain-based)
- Transaction/Log Entity
- Administrative Entity (if applicable)

4.7.2 Database Architecture

Explain:

- Client-Server Architecture

- Cloud Storage (if applicable)
- Distributed Storage (for Blockchain/ML)

4.8 Database Schema Diagram

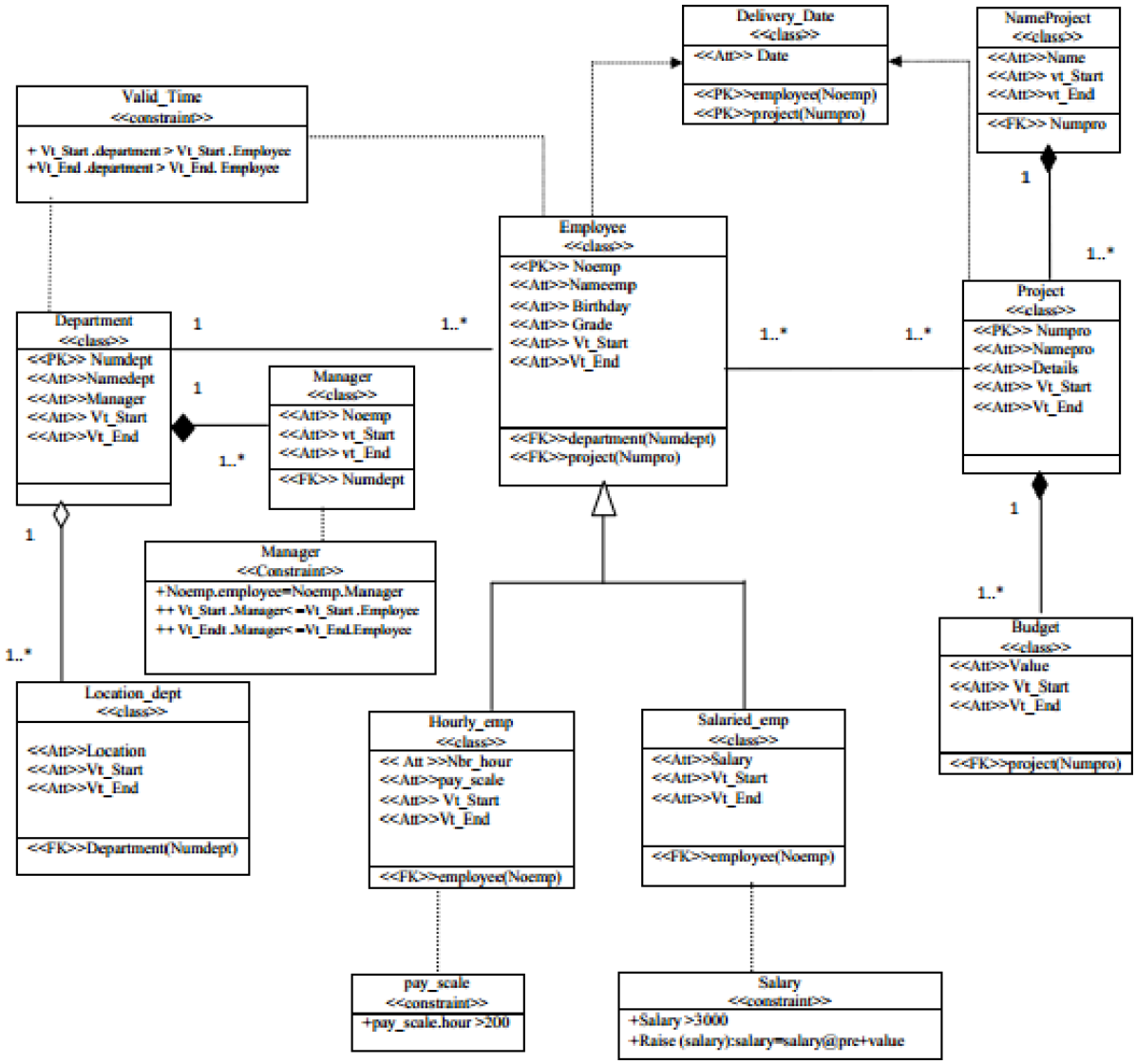

Figure 4.11: Sample Database Diagram

The Database Schema Diagram represents the physical database implementation structure.
Guidelines:

- Show actual table structure.
- Include column names and data types.
- Show PK and FK relationships.

- Reflect normalization level (1NF, 2NF, 3NF if applicable).

Explanation (50–80 words in final report): Explain how the schema supports:

- Data integrity
- Efficient querying
- Scalability
- Security

4.8.1 Data Dictionary

Data Dictionary defines structure of all data elements.

Guidelines:

- Include field name
- Data type
- Size
- Description
- Constraints

Table 4.1: Sample Data Dictionary – User Table

Field Name	**Type**	**Size**	**Description**
user_id	INT	11	Unique identifier for each user
username	VARCHAR	100	Name chosen by user
email	VARCHAR	150	Registered email address
password	VARCHAR	255	Encrypted password
role	VARCHAR	50	User type (Admin/User)

4.9 Database Tables / Collections

This section describes the logical database design of the system. It presents the major relational tables (for SQL-based systems) or collections (for NoSQL-based systems). The database structure is designed to ensure data consistency, integrity, security, and scalability.

Each table or collection includes primary keys, relationships, and major attributes relevant to system functionality.

4.9.1 User Collection / Table

The User table stores authentication credentials and user profile information. It supports role-based access control and secure authentication.

Description:

- Primary Key: `user_id`
- Relationships: Linked to transactions, predictions, or system logs.
- Authentication Attributes: Email, password hash, role, account status.

Table 4.2: User Table Structure

Field Name	Data Type	Constraint	Description
user_id	INT / UUID	Primary Key	Unique identifier for each user
full_name	VARCHAR(100)	Not Null	Stores user's full name
email	VARCHAR(150)	Unique, Not Null	User login email address
password_hash	VARCHAR(255)	Not Null	Encrypted password using hashing algorithm
role	VARCHAR(50)	Not Null	Defines user access level (Admin, User, etc.)
created_at	TIMESTAMP	Default Current	Account creation timestamp
status	BOOLEAN	Default True	Indicates whether account is active

4.9.2 Core Data Collection

This subsection describes domain-specific core data tables or collections. These structures store primary operational data processed by the system.

Domain-Specific Examples:

For Machine Learning Systems:

- Dataset Collection – Stores training and testing datasets.
- Model Metadata Collection – Stores model version, hyperparameters, and evaluation metrics.
- Prediction Results Collection – Stores inference results and timestamps.

For Web-Based Systems:

- Orders Table – Stores customer purchase records.
- Products Table – Stores product details and pricing.
- Transactions Table – Stores payment and transaction history.

For Mobile Applications:

- Device Info Table – Stores device metadata.
- Session Table – Tracks active user sessions.

For Blockchain-Based Systems:

- Transaction Collection – Stores blockchain transaction records.
- Wallet Collection – Stores wallet identifiers and balances.
- Block Metadata Collection – Stores block hash, timestamp, and validation status.

4.9.3 Relationships Between Tables

Database relationships define how data entities are connected within the system.

- **One-to-One (1:1):** Each user may have one profile detail record.
- **One-to-Many (1:N):** One user may generate multiple transactions or predictions.
- **Many-to-Many (M:N):** Users may interact with multiple products, and products may belong to multiple categories (resolved using junction tables).

An Entity-Relationship (ER) Diagram should be included to visually represent these relationships.

4.9.4 Security and Data Integrity

To ensure confidentiality and integrity of stored data, the following measures are implemented:

- Password Encryption using secure hashing algorithms (e.g., bcrypt, SHA-256 with salt).
- Role-Based Access Control (RBAC) to restrict unauthorized access.
- Input validation and constraint enforcement at database level.
- Secure communication using HTTPS.
- Regular database backup and disaster recovery strategy.

4.9.5 Scalability Considerations

The database design incorporates scalability mechanisms to support future growth.

- Horizontal Scaling using distributed database systems (if required).
- Indexing on frequently queried attributes to improve performance.
- Caching strategies for frequently accessed data.
- Load balancing across database instances.
- Partitioning or sharding for large-scale data environments.

4.10 Chapter Summary

This chapter presented the comprehensive software design of the proposed system. It translated the functional and non-functional requirements identified earlier into structured architectural, behavioral, and data models.

The design methodology and selected software process model were justified to ensure systematic development and maintainability. A high-level system overview was provided through the context diagram and architectural representation, clearly illustrating component interactions and subsystem boundaries.

Detailed UML models including activity diagrams, class diagrams, sequence diagrams, and state transition diagrams were developed to describe system behavior and structure. Additionally, data flow diagrams were constructed to represent internal data movement across different system layers.

The chapter further elaborated the database design through Entity Relationship Diagrams, database schema diagrams, and data dictionaries to ensure data integrity, scalability, and security. Relationships between entities and normalization considerations were also addressed.

Overall, this chapter establishes a clear technical blueprint that guides the implementation phase discussed in the next chapter.

Chapter 5
Implementation

This chapter describes the practical implementation of the proposed system. It explains how the designed architecture was translated into working modules, algorithms, user interfaces, and deployment components.

Implementation details may vary depending on the project domain (Machine Learning, Web, Mobile App, Blockchain, etc.).

5.1 Development Environment

This section describes the software and hardware environment used for system implementation.

Guidelines: Describe:

- IDE used (e.g., VS Code, Android Studio, PyCharm)
- Programming language(s)
- Framework(s) used
- Database system
- Operating system
- Hardware specifications (RAM, GPU if ML-based)

Write your content here.

5.2 Core Module Implementation

This section explains how each major module was implemented.

Guidelines: For each module:

- Describe its purpose
- Explain key functionalities
- Mention important classes/functions
- Explain integration with other modules

5.2.1 Module 1: [Module Name]

Explain implementation logic here.

5.2.2 Module 2: [Module Name]

Explain implementation logic here.

5.3 Algorithm Implementation

This section explains the implementation of the core prediction algorithm used in the system. The proposed system uses a Random Forest Classification algorithm to perform prediction tasks. Random Forest was selected due to its robustness, high accuracy, and ability to handle large feature sets with reduced overfitting.

The algorithm processes structured input data, performs feature selection, and generates classification outputs based on ensemble decision trees.

Guidelines:

- Explain algorithm logic
- Mention input and output
- Provide pseudo-code or actual code snippet
- Explain how results are generated

5.3.1 Algorithm 1: Random Forest Classification

Input: Preprocessed feature vector X
Output: Predicted class label Y

Step-by-Step Working:

1. Load the preprocessed dataset.
2. Split the dataset into training and testing sets.
3. Initialize Random Forest with n decision trees.
4. For each tree:
    - Randomly select a subset of training data (bootstrap sampling).
    - Randomly select a subset of features.
    - Build a decision tree using selected data.
5. Combine predictions from all trees using majority voting.
6. Output the final predicted class.

Pseudo Code:

```
Algorithm RandomForestClassifier(X, Y):

1. Split dataset into Train and Test
2. For i = 1 to N_trees:
      a. Create bootstrap sample
      b. Select random feature subset
      c. Train decision tree
3. Aggregate predictions using majority vote
4. Return final predicted class
```

Explanation:

The algorithm improves prediction performance by combining multiple weak learners (decision trees). Each tree is trained independently on randomly selected samples and features. This reduces variance and prevents overfitting. The final prediction is obtained by majority voting among all trees.

This algorithm integrates with the system backend and is triggered when a user submits input data for prediction.

5.3.2 Algorithm 2: [Name]

Input: Describe input
Output: Describe output

Explain step-by-step working.

Algorithm 1 Proof of Work Consensus

Input: Block header H, Target difficulty D
Output: Nonce value N such that $hash(H||N) < D$
function ProofOfWork($block\_header, target\_difficulty$)
    $nonce \leftarrow 0$
    while True **do**
        ▷ Concatenate the block header with the nonce
        $data \leftarrow block\_header \,||\, str(nonce)$
        ▷ Compute hash of concatenated data
        $hash\_value \leftarrow hash\_function(data)$
        ▷ Check if hash meets target difficulty
        if $hash\_value < target\_difficulty$ **then**
            return $nonce$ ▷ Valid nonce found
        else
            $nonce \leftarrow nonce + 1$ ▷ Increment nonce
        end if
    end while
end function

5.4 External APIs / SDKs

Guidelines: Describe:

- Third-party APIs used
- SDK integrations
- Purpose of usage
- Authentication mechanisms

Table 5.1: External APIs Used

API/SDK Name	**Purpose**	**Used In Module**
API 1	Detailed description of API functionality, integration purpose, authentication mechanism, and how it supports system operations.	User Management Module
API 2	Explanation of service provided, data exchange format, security handling, and response processing.	Payment Module

5.5 User Interface Implementation

This section presents a detailed explanation of all user interface screens developed for the system. Each screen is discussed in terms of its purpose, functionalities, navigation flow, validations, and integration with backend modules.

Students are required to include screenshots of every major screen implemented in the system.

5.5.1 UI Design Approach

Guidelines (200–300 words):

Students must explain:

- Design principles used (Minimalistic, Material Design, Dark Mode, etc.)
- UI/UX methodology followed
- Wireframing tools used (Figma, Adobe XD, etc.)
- Consistency in color scheme and typography
- Accessibility considerations
- Responsive design strategy (Mobile-first, adaptive layout)

Write detailed explanation here.

5.5.2 Screen-wise Implementation

Each major screen must be explained using the following structure:

- **Purpose of Screen**
- **Main Components (Buttons, Forms, Lists, Cards)**
- **User Interaction Flow**
- **Backend/API Integration**
- **Validation and Error Handling**

5.5.2.1 Screen 1: Login Screen

Purpose: Describe why this screen exists and which users access it.
Components:

- Email field
- Password field
- Login button
- Forgot password link

Functional Flow: Explain what happens when user enters credentials.
Backend Integration: Explain authentication API call.
Validation: Explain required field validation and error messages.

5.5.2.2 Screen 2: Dashboard Screen

Purpose: Explain dashboard objective.
Components:

- Navigation bar
- Data cards
- Action buttons

Functional Flow: Explain how data loads dynamically.
Backend Integration: Explain API endpoints used.
Security Considerations: Explain role-based access control.

5.5.2.3 Screen 3: Create Post / Input Screen

Explain detailed functionality including:

- Form submission
- Data validation
- Database storage
- UI feedback

5.5.3 Navigation Flow Diagram

Students must include a navigation flow diagram showing how screens are connected.

5.5.4 Responsiveness and Performance

Guidelines (150–250 words):
Explain:

- Mobile and desktop adaptability
- Loading time optimization
- State management strategy

5.6 Deployment

Guidelines: Describe:

- Deployment platform (AWS, Vercel, Firebase, etc.)
- CI/CD if used
- Server configuration
- Mobile app deployment method

Table 5.2: Deployment Details

Component	**Platform**	**URL/Details**
Backend	AWS EC2	https://example-backend.com
Frontend	Vercel	https://example-frontend.com
Mobile App	Firebase Hosting	Play Store / TestFlight Link

5.7 Chapter Summary

This chapter presented the detailed implementation of the proposed system. The conceptual design and architectural models discussed in the previous chapter were translated into practical software components, including backend services, core functional modules, algorithmic engines, and user interface elements.

The development environment and configuration were described to ensure reproducibility of the implementation process. Each major module was implemented in a modular and loosely coupled manner to enhance maintainability and scalability. The core prediction algorithm was integrated into the system backend, enabling real-time processing of user input and generation of classification results.

External APIs and SDK integrations were discussed where applicable, highlighting authentication mechanisms and secure communication practices. The user interface implementation was explained screen-by-screen, including validation logic, navigation flow, and backend integration.

Finally, deployment strategies were presented, detailing hosting platforms, server configuration, and production setup. Overall, this chapter demonstrates the successful realization of the proposed system into a functional, deployable solution ready for testing and evaluation.

Chapter 6
Testing and Evaluation

This chapter presents the testing strategies and evaluation procedures used to verify the correctness, reliability, and performance of the implemented system.

Testing ensures that all functional and non-functional requirements defined in earlier chapters are satisfied.

Depending on the project domain (Machine Learning, Web, Mobile Application, Blockchain, etc.), evaluation methods may vary.

6.1 Testing Strategy

Guidelines (150–200 words):

In this section:

- Describe the overall testing approach
- Mention testing types used (Unit, Integration, System)
- Explain manual vs automated testing
- Mention tools used (JUnit, Selenium, PyTest, etc.)

Write detailed explanation here.

6.2 Unit Testing

Unit testing verifies individual modules independently.

Guidelines:

- Each major function/module must be tested
- Provide input, expected output, and result
- Mention Pass/Fail status

6.2.1 UT-01: User Registration

Table 6.1: Unit Test Case – User Registration

Test ID	Input	Expected Output	Result
UT-01	Valid user credentials	Account created successfully	Pass
UT-02	Missing email field	Error message displayed	Pass

6.2.2 UT-02: Login Module

Add more unit test cases similarly.

6.3 Integration Testing

Integration testing ensures that modules interact correctly.

Guidelines (150–200 words):

- Describe module interaction testing
- Explain API testing if applicable
- Mention database connectivity validation

Table 6.2: Integration Testing Summary

Test ID	Modules Integrated	Expected Result	Result
IT-01	Frontend + Backend API	Data successfully retrieved	Pass
IT-02	Backend + Database	Record stored correctly	Pass

6.4 System Testing

System testing validates the entire application workflow.

Guidelines:

- Test full user journey
- Test edge cases
- Test error handling

Describe system-level validation here.

6.5 Performance Testing

Guidelines (100–150 words):

- Mention response time
- Load handling capability
- Stress testing (if applicable)

Table 6.3: Performance Metrics

Metric	Description	Value
Response Time	Average time per request	220 ms
Concurrent Users	Maximum supported users	500

6.6 Security Testing

Guidelines:

- Authentication validation
- Authorization testing
- SQL injection testing
- Data encryption validation

Explain security verification here.

6.7 Model Evaluation (For ML-Based Projects)

This section is required only for Machine Learning projects.

Guidelines (200–300 words):

- Mention dataset split (Train/Test)
- Explain evaluation metrics
- Present confusion matrix
- Compare baseline models

6.7.1 Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1-Score

Figure 6.1: Confusion Matrix

Table 6.4: Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score
CNN	94%	92%	93%	92.5%
Random Forest	89%	88%	87%	87.5%

6.8 User Acceptance Testing

Guidelines:

- Mention number of users tested
- Feedback summary
- Improvements suggested

Table 6.5: User Feedback Summary

Feedback	Action Taken
UI improvement needed	Navigation simplified
Performance delay observed	Backend optimized

6.9 Testing Summary

Provide a concise summary of:

- Total test cases executed
- Number of passed cases
- System reliability
- Final system readiness

Conclude this chapter with validation statement that the system meets functional and non-functional requirements.

Chapter 7
Conclusion and Future Work

7.1 Conclusion

Guidelines (300–500 words): Summarize:

- Problem addressed
- System solution
- Key achievements
- Evaluation results
- Overall contribution

Write your conclusion here.

7.2 Future Work

Guidelines (200–300 words): Mention:

- System improvements
- Scalability enhancements
- AI/ML improvements
- Security enhancements
- Additional modules

Write future work here.

7.3 Limitations

Briefly mention system limitations:

- Data limitations
- Infrastructure limitations
- Model constraints

Bibliography

[1] M. Ullah, H. Qiu, W. Huangfu, D. Yang, Y. Wei, and B. Tang, "Integrated machine learning approaches for landslide susceptibility mapping along the pakistan–china karakoram highway," *Land*, vol. 14, no. 1, 2025. [Online]. Available: https://www.mdpi.com/2073-445X/14/1/172

[2] A. Rehman, J. Song, F. Haq, S. Mahmood, M. I. Ahamad, M. Basharat, M. Sajid, and M. S. Mehmood, "Multi-hazard susceptibility assessment using the analytical hierarchy process and frequency ratio techniques in the northwest himalayas, pakistan," *Remote Sensing*, vol. 14, no. 3, 2022. [Online]. Available: https://www.mdpi.com/2072-4292/14/3/554

Appendices

Appendix A - Turnitin Similarity Report

This appendix contains the official Turnitin Similarity Report generated for the final submitted version of this dissertation.

Plagiarism Report Screenshot:

Appendix B - AI Writing Detection Report

This appendix contains the AI Writing Detection Report generated by Turnitin (if applicable), verifying compliance with institutional academic integrity guidelines regarding AI-assisted content.

AI Detection Report Screenshot:

Appendix C – Source Code

This section provides details about the project source code repository.

Guidelines for Students:

- Upload your complete project source code to a version control platform such as GitHub, GitLab, or Bitbucket.
- Ensure the repository includes:
  - All source files
  - Database scripts (if applicable)
  - Model training and testing code (for ML projects)
  - Frontend and backend code (for web/mobile projects)
  - README file with setup instructions
- The repository must be properly structured with meaningful folder names.
- Remove unnecessary files (temporary files, cache files, datasets if restricted).
- If the repository is private, make the repository public before final submission.
- Paste the final repository link below.

Repository Link:

`PasteYourGitHubRepositoryLinkHere`

Additional Notes:

- Mention branch name if different from `main`.
- Mention if any third-party APIs require API keys.
- Mention hardware requirements (if ML/GPU-based project).